uc3m | Universidad Carlos III de Madrid

Master Degree in
Applied and Computational Mathematics
2025-2026

Master Thesis

Computational algorithms for pricing financial derivatives under local volatility models

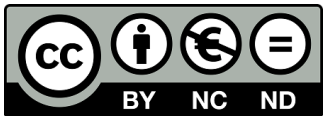
---

Jose Enrique Zafra Mena

Jorge Morón Vidal
Francisco Manuel Bernal Martínez

Madrid, 2026

AVOID PLAGIARISM
The University uses the **Turnitin Feedback Studio** for the delivery of student work. This program compares the originality of the work delivered by each student with millions of electronic resources and detects those parts of the text that are copied and pasted. Plagiarizing in a TFM is considered a **Serious Misconduct**, and may result in permanent expulsion from the University.



SUMMARY

Keywords:

DEDICATION

CONTENTS

LIST OF FIGURES

LIST OF TABLES

1. INTRODUCTION

1.1. Motivation

- emepezar el capítulo explicando el contexto práctico de valoración de derivados bajo volatilidad local y por qué importa que los métodos sean eficientes y numéricamente estables
- dejar claro el problema principal: los métodos muy precisos suelen ser caros en calibración o en lotes grandes de precios, y eso limita su uso
- justificar por qué compensa explorar modelos sustitutos y aproximaciones neuronales para reducir coste computacional sin perder control del error
- aclarar desde el principio el caso de uso que tengo en mente (prototipo de investigación frente a sistema listo para producción) para no prometer más de lo que realmente cubre el trabajo ver si la narrativa final se centra solo en opciones vanilla

1.2. Main Objectives

- el objetivo es diseñar y evaluar algoritmos computacionales para valorar derivados bajo modelos de volatilidad local, teniendo en cuenta el equilibrio entre velocidad y precision
- objetivo 1: montar un pipeline reproducible de datos, configuración de modelos y ejecución de experimentos para poder repetir benchmarks de forma consistente. actualizar con referencias exactas a scripts y módulos cuando cierre definitivamente el pipeline
- objetivo 2: desarrollar y comparar enfoques sustitutos (modelos neuronales, LTSM, etc) frente a referencias numéricas bajo condiciones de evaluación comunes
- objetivo 3: medir el rendimiento con métricas de error de precio, coste computacional y estabilidad en distintos regímenes de mercado y regiones de parámetros
- objetivo 4: evaluar limitaciones prácticas y fijar fronteras claras sobre qué parte está preparada para uso aplicado y qué parte sigue siendo exploratoria decidir si incluyo objetivos explícitos de latencia para inferencia y calibración en la versión final

1.3. Key Contributions

- 1-> presentar un marco experimental para comparar métodos numéricos clásicos y sustitutos neuronales en tareas de volatilidad local
- 2-> aportar evidencia de dónde los modelos sustitutos aceleran y dónde fallan al generalizar, incluyendo escenarios de estres añadir el conjunto final de escenarios de estrés cuando termine los experimentos pendientes de robustez
- 3-> recoger aprendizajes de implementación sobre estrategia de entrenamiento, control de error y criterios de selección de modelos que puedan reutilizarse en problemas parecidos
- 4-> discutir amenazas a la validez, como calidad de datos, sesgo de discretización, riesgo de extrapolación y dependencia del benchmark elegido incluir sensibilidad a costes de transacción y slippage solo si termino a tiempo la integración con el flujo de cartera
- 5-> cerrar con un roadmap futuro priorizado por impacto esperado y complejidad de implementación

1.4. Scope and Structure

- delimitar explícitamente el alcance, indicando qué cubro y qué dejo fuera, sobre todo en universo de activos, rango de vencimientos y supuestos de microestructura
- resumir la validación a alto nivel: datos usados, lógica de partición entre entrenamiento, validación y test, y criterios para seleccionar benchmarks cerrar la lista de benchmarks y el periodo de datos después de la última revisión de calidad
- cerrar la introducción con una guía breve capítulo a capítulo para que se entienda rápido el recorrido desde fundamentos teóricos hasta métodos, resultados y conclusiones

2. MATHEMATICAL AND FINANCIAL BACKGROUND

2.1. Mathematical Framework For Option Pricing

- hablar del marco matematico general de valoracion de opciones para que el resto del capitulo tenga una base comun.
- comentar de forma explicita la notacion de tiempos, activos, variables de estado y parametros para evitar ambiguedades en capitulos posteriores.
- dejar claro, breve, la idea de medida neutral al riesgo y por que el precio se expresa como esperanza descontada del payoff (Feynman-Kac)
- cerrar la seccion conectando este marco general con los modelos concretos que uso despues (heston y formulacion con ChF)

decidir sobre si incluir aqui una derivacion formal completa o dejar detalles tecnicos para un apendice
ver si me conviene incluir una tabla corta de simbolos para facilitar la lectura.

2.2. Stochastic Calculus Background

- recordar solo el calculo estocastico minimo necesario para entender las ecuaciones del modelo y no convertir esta parte en un curso completo.
- introducir primero objetos basicos (movimiento browniano, procesos ito, esperanza condicional) y luego pasar a funciones caracteristicas y cumulantes.
- mantener foco en la intuicion financiera de cada definicion para que no quede como una lista aislada de formulas.
- cerrar enlazando estos conceptos con el metodo cos que aparece mas adelante.

revisar consistencia de notacion entre esta seccion y la seccion de heston

TODO: add Cumulative Distribution Function definition
TODO: add Probability Density Function definition (and delete from below)

[Oorsterlee book] The *Characteristic Function* (ChF) for $u \in \mathbb{R}$ of the random variable X is the Fourier-Stieltjes transform of the cumulative distribution function $F_X(x)$.

$$\phi_X(u) := \mathbb{E}\left[e^{iuX}\right] = \int_{-\infty}^{\infty} e^{iux} dF_X(x) = \int_{-\infty}^{\infty} e^{iux} f_X(x) dx, \tag{2.1}$$

where $f_X(u)$ is the *Probability Density Function*, given by

$$f_X(u) = \frac{\partial F_X(u)}{\partial u}. \tag{2.2}$$

Applying the logarithm to the ChF, we define the *Cumulant Characteristic Function* by

$$\zeta_X(u) = \log \mathbb{E}\left[e^{iux}\right] = \log \phi_X(u). \tag{2.3}$$

If we compute the MacLaurin expansion to ζ_X, we arrive to

$$\zeta_X(u) = \sum_{k=0}^{\infty} \frac{\partial^k \zeta_k(u)}{\partial u^k}\bigg|_{u=0} \frac{u^k}{k!} = \sum_{k=0}^{\infty} \zeta_k \frac{u^k}{k!}, \tag{2.4}$$

where $\zeta_k \equiv \frac{\partial^k \zeta_X(u)}{\partial u^k}\Big|_{u=0}$ is the k-th *cumulant*.

Furthermore, we can define the *Moment Generating Function*, $\mathcal{M}_X(u)$, by

$$\mathcal{M}_X(u) := \phi_X(-iu) = \mathbb{E}\left[e^{ux}\right]. \tag{2.5}$$

Since the MGF is $\mathcal{M}_X(u) = \phi_X(-iu)$ and $\zeta_X(u) = \log \mathcal{M}_X(u)$, we can compute the cumulants, using the ChF, as

$$\zeta_k = \frac{\partial^k \zeta_X(u)}{\partial u^k}\bigg|_{u=0} = \frac{\partial^k}{\partial u^k} \log \phi_X(-iu)\bigg|_{u=0}. \tag{2.6}$$

2.3. The Heston Stochastic Volatility Model

- presentar la dinamica del modelo de heston, indicando que la volatilidad sigue proceso CIR (mean regresion y tal)
- explicar el papel de cada parametro $(\kappa, \bar{\nu}, \gamma, \rho)$ y como afecta a la forma de la sonrisa de IV
- anotar la condicion de feller y comentar cuando se cumple o no en calibraciones
- cerrar conectando el modelo con la ChFque necesito para aplicar la valoracion por fourier

anadir referencia bibliografica de heston (1993) y una referencia de implementacion numerica estable.

2.4. Pricing Via Characteristic Functions

- explicar por que pasar al dominio de fourier simplifica la valoracion cuando dispongo de ChF cerrada

- derivar el esquema general de precio usando transformadas y dejar claro la hipotesis de integrabilidad
- justificar la eleccion de esta via frente a alternativas como pde o monte carlo
- cerrar dejando preparada la transicion natural al metodo fourier-cos

comprobar si incluyo una comparativa breve de coste computacional teorico frente a metodos alternativos.

2.5. The Fourier-COS method

- presentar el metodo cos como aproximacion espectral de densidades y precios en un intervalo truncado
- comentar con cuidado como elegir el intervalo $[a, b]$ a partir de cumulantes y que impacto tiene esa eleccion en el error (un poco en apendice ya)
- separar claramente error de truncacion y error por numero finito de terminos para luego interpretar mejor los resultados numericos (tambien en apendice ahora)
- ir diciendo formula, significado de cada termino y como se implementa (vectorizado)

anadir como una guia practica para elegir N y evitar artefactos numericos en extremos de K y vencimiento

The COS method is the ...
The main reasons we want to use a cosine expansion are:

- In a cosine Fourier expansion we can represent even functions around 0 exactly.
- We can extend any function $g : [0, \pi] \to \mathbb{R}$ to become an even function on $[-\pi, \pi]$ as follows:

$$\bar{g}(\theta) = \begin{cases} g(\theta), & \theta \geq 0, \\ g(-\theta), & \theta < 0. \end{cases} \tag{2.7}$$

- For functions supported on any other finite interval, say $[a, b] \in \mathbb{R}$, the Fourier cos series expansion can be obtained via a change of variables:

$$\theta := \frac{y - a}{b - a}\pi, \quad y := \frac{b - a}{\pi}\theta + a.$$

TODO: falta más teoría para enlazar

We finally arrive to the approximation of the PDF

$$\hat{f}_X(y) = \sum_{k=0}^{N-1}{}' \bar{F}_k \cos\left(\pi k \frac{y-a}{b-a}\right), \tag{2.8}$$

where $\sum'$ indicates that the first term should be multiplied by $1/2$, and the coefficients depend on the ChF:

$$\bar{F}_k := \frac{2}{b-a} \operatorname{Re}\left[\phi_X\left(\frac{\pi k}{b-a}\right) \cdot \exp\left(-\frac{i\pi a k}{b-a}\right)\right]. \tag{2.9}$$

TODO: no se si poner algo aquí o nueva subsection

The value of a plain vanilla option is given by

$$V(t_0, x) = e^{-r\tau} \int_{\mathbb{R}} V(T, y) f_X(T, y; t_0, x) dy, \tag{2.10}$$

where $\tau = T - t_0$, $f_X(T, y; t_0, x)$ is the transition probability density of $X(t)$, and r the interest rate.

Applying the COS method to the last expression, we arrive to

$$V(t_0, x) \approx e^{-r\tau} \sum_{k=0}^{N-1}{}' \operatorname{Re}\left[\phi_X\left(\frac{\pi k}{b-a}\right) \cdot \exp\left(-\frac{i\pi a k}{b-a}\right)\right] \cdot H_k, \tag{2.11}$$

with x a function of $S(t_0)$, $\phi_X(u)$ the ChF, and H_K the *payoff coefficients*:

$$H_k := \frac{2}{b-a} \int_a^b V(T, y) \cos\left(\frac{y-a}{b-a}\pi k\right) dy. \tag{2.12}$$

TODO: seguir con los payoffs coeffients

2.5.1. The COS Method Applied To The Heston Model

TODO: rellenar

- presentr la teoria anterior al caso heston, con definiciones consistentes de variables y convenciones de signos
- verificar que la expresion de φ_H que uso coincide con una formulacion estandar y numericamente estable
- dejar claro como paso de la formula escalar a evaluacion vectorizada en mallas de strikes y vencimientos

- cerrar con una lista corta de sanity checks numericas antes de usar esta formula en benchmarks

revisar simbolos de entrada/salida en esta subseccion para que coincidan exactamente con la implementacion en codigo.

We need to define the following functions:

$$D_1(u) = \sqrt{(\kappa - \gamma\rho i u)^2 + (u^2 + iu)\,\gamma^2}, \tag{2.13}$$

$$g(u) = \frac{\kappa - \gamma\rho i u - D_1}{\kappa - \gamma\rho i u + D_1}. \tag{2.14}$$

Using a vector of strikes $\mathbf{K}$, we find the COS formula for the Heston model:

$$V(t_0, \mathbf{x}) \approx \mathbf{K} e^{-r\tau} \cdot \mathrm{Re}\left\{ {\sum_{k=0}^{N-1}}' \varphi_H\left[\frac{\pi k}{b-a}, T; \nu(t_0)\right] U_k \cdot \exp\left[i\pi k \frac{\mathbf{X}(t_0) - a}{b-a}\right]\right\}, \tag{2.15}$$

where no se si es la ChF?

$$\begin{aligned} \varphi_H(t, T; \nu(t_0)) = \exp&\left\{ iur\tau + \frac{\nu(t_0)}{\gamma^2}\left(\frac{1 - e^{-D_1\tau}}{1 - g e^{-D_1\tau}}\right)(\kappa - i\rho\gamma u - D_1)\right\} \cdot \\ &\cdot \exp\left\{\frac{\kappa\bar{\nu}}{\gamma^2}\left(\tau(\kappa - i\rho\gamma u - D_1) - 2\log\left(\frac{1 - g e^{-D_1\tau}}{1 - g}\right)\right)\right\}. \end{aligned} \tag{2.16}$$

2.6. Implied Volatility And Numerical Inversion

- explicar como paso de precios a volatilidad implicita y por que esta inversion es clave para comparar modelos.
- documentar el metodo numerico de inversion que uso (por ejemplo, newton o brent), incluyendo tolerancias y criterios de parada.
- comentar problemas tipicos de inversion (no convergencia, soluciones no fisicas, sensibilidad a la semilla) y como los trato.
- cerrar definiendo claramente que metricas en volatilidad implicita voy a reportar en resultados.

decidir si reporto errores en precio, en iv o ambos segun el objetivo de cada experimento.

2.7. State Of The Art

- resumir el estado del arte en tres bloques: metodos clasicos de valoracion, metodos espectrales y modelos sustitutos basados en redes neuronales.

- destacar no solo resultados positivos sino tambien limites reportados (generalizacion, estabilidad fuera de distribucion y coste de entrenamiento).
- posicionar mi trabajo de forma explicita: que parte reutiliza ideas existentes y que parte aporta una comparacion o implementacion nueva.
- cerrar con una tabla comparativa breve que justifique las decisiones metodologicas de los capitulos siguientes.

cerrar el set final de referencias recientes y priorizar articulos con codigo reproducible.

3. NEURAL NETWORK PRICER FOR HESTON VANILLA OPTIONS

The main objective of this chapter is to construct an ANN-based surrogate pricer for European vanilla options under the Heston stochastic-volatility model. The target quantity is implied volatility, and the surrogate is designed to approximate the forward map

$$(\Theta, m, \tau, r) \mapsto \sigma_{imp}.$$

Reference pricing based on numerical methods (in our pipeline, Heston COS valuation plus implied-volatility inversion) is highly accurate but computationally expensive when repeated many times, as required in calibration and sensitivity workflows. The ANN aims to preserve pricing accuracy while reducing inference time by several orders of magnitude.

Methodologically, this chapter follows the ANN-pricing framework of Liu et al. [1] as the main reference (architecture family, data ranges, and training philosophy), while explicitly documenting the implementation choices adopted in this project.

This chapter focuses exclusively on the pricing network; the calibration architecture is treated separately in Chapter 4.

3.1. Problem Formulation

In this work, vanilla option pricing under Heston is posed as a supervised regression problem. Given a dataset

$$\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^{N}, \qquad y_i = \sigma_{imp,i},$$

the neural network approximates the map

$$f_{NN} : \mathbb{R}^8 \to \mathbb{R}, \qquad \mathbf{x} = (\rho, \kappa, \gamma, \bar{v}, v_0, m, \tau, r) \mapsto \sigma_{imp},$$

where $m = S_0/K$ is moneyness and $\tau = T - t$ is time-to-maturity. Each label σ_{imp} is obtained from Heston-consistent synthetic prices through numerical inversion.

The training objective is to estimate parameters $\mathbf{w}$ such that the empirical prediction error over $\mathcal{D}$ is minimized. In this chapter, we use the ANN only as a pricing surrogate, i.e., as a fast approximation of the reference numerical pipeline in the forward map $(\Theta, m, \tau, r) \mapsto \sigma_{imp}$.

We learn implied volatility instead of option price because the IV representation is directly comparable across contracts with different maturities and moneyness levels, and it is the market-standard quotation format for calibration and diagnostics. In contrast, raw

prices mix scale effects (spot/strike/time) that make learning less homogeneous over the domain.

The resulting surrogate is primarily an in-domain approximator. Preliminary out-of-domain checks indicate that the network remains stable for moderate departures from the training bounds, while errors increase in extreme regimes. For this reason, the methodological core of this chapter is developed on the predefined training domain, and the extrapolation behaviour is discussed separately in the generalization and robustness analysis.

3.2. Synthetic Data Formulation

Training requires a large set of labeled input-output pairs. To obtain a controlled and fully reproducible dataset, we use synthetic data generated under the Heston model. For each sampled input vector, the target implied volatility is computed through the numerical pricing pipeline (COS valuation followed by implied-volatility inversion).

In the current pipeline, data generation is organized as a *master dataset* shared across downstream models. Each row stores:

- features: $(\rho, \kappa, \gamma, \bar{v}, v_0, m, \tau, r)$,
- targets: $(price_{\cos}, \sigma_{\text{imp}}^{\text{brent}})$,
- quality labels: $(feller\_slack, feller\_ok, brent\_success, brent\_abs\_residual)$.

This design allows ANN and PINN workflows to reuse the same synthetic source while applying experiment-specific filters only at training time.

The raw variable set includes Heston parameters $\Theta = \{\rho, \kappa, \gamma, \bar{v}, v_0\}$ and contract/market variables $\Sigma = \{K, S_0, \tau, r\}$, which defines a 9-dimensional space. In our setup, we fix $K = 1$ and represent the underlying level through moneyness $m = S_0/K$, leading to an 8-dimensional input vector:

$$\mathbf{x} = (\rho, \kappa, \gamma, \bar{v}, v_0, m, \tau, r) \in \mathbb{R}^8.$$

A full Cartesian grid is computationally prohibitive in this dimension, since using N points per variable scales as N^9 (already intractable for moderate N). Therefore, we rely on Latin Hypercube Sampling (LHS) to obtain broad coverage of the domain with a fixed number of samples. The specific ranges, constraints, and splitting/preprocessing rules are detailed in the following subsections.

3.2.1. Parameter Ranges And Constraints

The sampling domain combines Heston parameters, contract variables, and market rate. Following the ranges used in our configuration (largely aligned with [1]), the training

domain is summarized in the table below.

TABLE 3.1. INPUT RANGES USED FOR SYNTHETIC DATA GENERATION

Parameter	Range
ρ	$[-0.9, 0.0]$
κ	$[0.0, 3.0]$
γ	$[0.01, 0.8]$
$\bar{v}$	$[0.01, 0.5]$
v_0	$[0.05, 0.5]$
m	$[0.6, 1.4]$
τ	$[0.05, 3.0]$
r	$[0.0, 0.05]$

Source: Based on [1] and project configuration choices

To avoid non-physical or numerically unstable samples, we enforce $\tau > 0$ and monitor a Feller-type admissibility condition

$$2\kappa\bar{v} - \gamma^2 \geq 0,$$

through the scalar label

$$feller\_slack = 2\kappa\bar{v} - \gamma^2,$$

and the binary indicator $feller\_ok = \mathbb{1}\{feller\_slack \geq 0\}$. Importantly, Feller-violating points are *not* removed during generation; they are kept in the master dataset for controlled ablation studies.

During synthesis we fix the strike to $K = 1$ and price European put options. Fixing K is equivalent to expressing spot through moneyness, which reduces the dimensionality from nine variables $(\Theta, S_0, K, \tau, r)$ to eight inputs (Θ, m, τ, r) without loss of information for this setup.

3.2.2. Sampling Strategy: Latin Hypercube Sampling

The training of the neural network requires generating a large set of model parameters distributed over a high-dimensional domain. In our case, the parameter space is $\mathbb{R}^8$. A purely random sampling of this space may generate highly populated regions while leaving other regions scarcely explored. As a consequence, the resulting training dataset may be poorly representative, leading to suboptimal training performance.

To avoid this problem, we will use the *Latin Hypercube Sampling* (LHS), a sampling method designed to uniformly cover each dimension. The main idea consists in splitting each dimension of the parameter space into N disjoint subintervals of equal length, and randomly selecting exactly one sample within each subinterval.

Thus, LHS guarantees a one-dimensional marginal distribution along each coordinate, independently of the total dimension of the space. This results in a sampling scheme that is more representative of the parameter space.

This creates a representative distribution sampling of the parameter space. In contrast to a full-grid sampling, the total number of samples N remains fixed, preventing exponential growth in the number of dimensions.

The use of LHS is supported by the classical design-of-experiments literature, where stratified one-dimensional coverage is shown to improve space-filling behaviour with respect to naive random sampling for a fixed budget [2]. In our implementation, sampling is performed directly in the 8-dimensional input domain with a fixed seed and a target of 10^7 samples in the master dataset. This is consistent with standard surrogate-modeling practice, where the quality and coverage of the design points strongly influence final emulator accuracy and robustness [3].

3.2.3. Dataset Splitting And Preprocessing

After generation, we keep all selected rows in the master dataset and split into train/-validation/test subsets with proportions $0.8/0.1/0.1$. In the reported configuration, splits are random (without mandatory stratification), and filtering policies are deferred to each training experiment.

In the baseline configuration, no explicit feature or target normalization is applied (Liu-style setup). Therefore, the model is trained directly on the physical scale of the variables $(\rho, \kappa, \gamma, \bar{v}, v_0, m, \tau, r)$ and predicts IV in absolute units.

Data quality control is integrated in the synthesis pipeline through explicit labels, not hard row deletion by default. In particular, we track Brent inversion status and reconstruction error

$$|V_{BS}(\sigma_{imp}) - V_{\text{target}}|,$$

while keeping all generated points available for later filtering. For storage efficiency, output tables are persisted in `float32`, whereas COS pricing and Brent inversion are computed internally in `float64`.

3.2.4. Numerical Label Generation (COS + Brent)

Each label is produced in two deterministic numerical steps. First, the target option value V_{target} is computed with the Heston COS solver for a given input vector $(\rho, \kappa, \gamma, \bar{v}, v_0, m, \tau, r)$. Second, implied volatility is obtained by solving

$$V_{BS}(\sigma) = V_{\text{target}}$$

with a Brent root-finding routine under fixed bounds and tolerance [4].

In our implementation, Brent is configured with strict numerical controls (absolute tolerance 10^{-8}, bounded search interval, and iteration cap). Additionally, when the inferred volatility hits the lower IV floor, an adaptive COS fallback can be triggered: the truncation width is increased ($L \leftarrow 1.5L$, with optional N scaling) and inversion is retried. The retry is accepted only if it improves numerical quality (e.g., leaves the floor or reduces reconstruction error).

Rather than dropping those edge cases at generation time, the pipeline records quality metadata (`brent_success`, `brent_abs_residual`, and retry diagnostics) so that robustness filters can be applied transparently at training time. This preserves full traceability of numerical reliability across the domain.

3.3. Neural Network Architecture

The pricing surrogate is implemented as a fully connected neural network that approximates a smooth nonlinear map from model/contract inputs to implied volatility. This choice is consistent with the structure of the problem: inputs are low-dimensional continuous features without spatial topology or sequential dependence, and the target is a scalar regression value.

From a function-approximation perspective, a feed-forward multilayer perceptron (MLP) provides enough expressive capacity to capture the interactions between Heston parameters, moneyness, maturity, and rates on the selected domain. Compared with architectures designed for images or sequences (e.g., CNNs or transformers), an MLP is more parsimonious and easier to train for this tabular setting.

3.3.1. Model Class

Following [1], we use a fully connected feed-forward network (FCNN). Let $\mathbf{x} \in \mathbb{R}^8$ be

$$\mathbf{x} = (\rho, \kappa, \gamma, \bar{v}, v_0, m, \tau, r),$$

and let $y = \sigma_{imp} \in \mathbb{R}$ denote the target implied volatility. The model defines a parametric function

$$\hat{\sigma}_{imp} = f_{NN}(\mathbf{x}; \mathbf{w}),$$

with trainable parameters $\mathbf{w}$.

Once $\mathbf{w}$ is fixed after training, inference is deterministic: for the same input $\mathbf{x}$, the network always returns the same prediction $\hat{\sigma}_{imp}$. This property is important for downstream calibration and sensitivity analysis, where numerical consistency across repeated calls is required.

The FCNN acts as a surrogate only at inference time. Target labels are still generated with the numerical pipeline (Heston COS pricing followed by implied-volatility inversion), so the network learns to emulate that reference solver on the sampled domain.

3.3.2. Architecture Details

The architecture used in this chapter is

$$8 \to 200 \to 200 \to 200 \to 200 \to 1,$$

with ReLU activation in hidden layers and linear activation in the output layer. In compact form,

$$h^{(1)} = \phi(W^{(1)}x + b^{(1)}), \quad \ldots, \quad h^{(4)} = \phi(W^{(4)}h^{(3)} + b^{(4)}), \quad \hat{y} = W^{(5)}h^{(4)} + b^{(5)},$$

where $\phi(\cdot) = \max(0, \cdot)$.

Weights are initialized with Xavier uniform initialization and biases are initialized to zero. In the baseline setup, dropout is disabled ($p = 0$) and no batch normalization layers are used. This configuration follows a controlled, reproducible design intended to isolate the impact of optimization and data quality choices.

TABLE 3.2. NEURAL NETWORK ARCHITECTURE USED FOR THE ANN PRICER

Hyperparameter	**Value**
Input dimension	8
Hidden layers	4
Hidden width	200 neurons per layer
Hidden activation	ReLU
Output dimension	1
Output activation	Linear
Weight initialization	Xavier uniform
Bias initialization	Zeros
Dropout	$p = 0.0$
Batch normalization	Disabled

Source: Based on [1]

3.3.3. Output Interpretation

The network output is a scalar implied volatility in annualized units (decimal form), not an option price. Therefore, prediction errors are interpreted directly in IV space, which is the representation used in the calibration workflow and in most volatility-surface diagnostics.

Because the output layer is linear, the model is not hard-constrained to produce strictly positive values for every possible input. In practice, training on a physically admissible domain with positive IV labels strongly reduces this risk inside the sampled region. For downstream use, any out-of-domain prediction can be explicitly checked and, if required, filtered before subsequent numerical steps.

This output definition also preserves differentiability of the surrogate with respect to inputs, which is useful in later chapters when the ANN pricer is embedded in calibration and sensitivity pipelines.

3.4. Training Procedure

Training is fully configuration-driven and follows an epoch-based supervised learning loop. At each epoch, the active optimizer performs parameter updates on the training split, then the model is evaluated on the validation split without gradient computation. For Adam updates, optimization is done with standard mini-batch backpropagation; for L-BFGS updates, each step is computed through a closure that evaluates the loss and its gradient on the selected batch regime.

Each epoch stores train/validation metrics, current optimizer identity, and learning rate. When enabled, the validation monitor can be either the raw validation loss or a trimmed variant that discards the top-tail hardest samples before aggregation. This provides a robust alternative when a small fraction of outlier points may dominate the monitor value.

Execution device is selected automatically (`mps` when available, otherwise `cpu`), and data-shuffling reproducibility is controlled through a fixed generator seed from configuration. During training, two checkpoints are maintained: the *last* checkpoint (always overwritten each epoch) and the *best* checkpoint (updated only when the monitored validation metric improves). The best checkpoint is used as the reference model for post-training evaluation and downstream calibration.

3.4.1. Loss Function

Let $\{(\mathbf{x}_i, \sigma_i)\}_{i=1}^{B}$ denote a mini-batch, where σ_i is the target implied volatility and $\hat{\sigma}_i = f_{NN}(\mathbf{x}_i; \mathbf{w})$. The baseline objective is mean squared error (MSE):

$$\mathcal{L}_{\text{MSE}}(\mathbf{w}) = \frac{1}{B} \sum_{i=1}^{B} (\hat{\sigma}_i - \sigma_i)^2 .$$

MSE is preferred because it is smooth, differentiable everywhere, and strongly penalizes large deviations, which is desirable when the ANN is later used in calibration loops sensitive to local pricing errors. Although MAE and RMSE are available in the implementation, MSE is used as the main training criterion in the reported baseline.

3.4.2. Optimizer And Hyperparameters

PUEDE IR CAMBIANDO: se pretende mejorar las runs

Several optimizer configurations were tested (Adam, mixed schedules, and `mix_half`). For model selection, we prioritize out-of-sample absolute-error behaviour (MAE and quantile stability), since these metrics better reflect the typical pointwise error level used later in calibration and sensitivity tasks.

Under this criterion, the selected reference ANN is `Liu_like_v01` (Adam), while `MIX_half_v01` is the second-best candidate. Although `MIX_half_v01` achieves a slightly lower validation MSE, `Liu_like_v01` provides lower validation and test MAE, together with better high-quantile absolute errors.

TABLE 3.3. REFERENCE AND RUNNER-UP RUNS USED IN OPTIMIZER SELECTION

Run	Optimizer mode	Val MAE	Test MAE
`Liu_like_v01`	Adam	4.1132×10^{-4}	4.2925×10^{-4}
`MIX_half_v01`	Adam → L-BFGS	5.1342×10^{-4}	5.2377×10^{-4}

Source: Own elaboration from `outputs/runs/*/metrics/eval_global.csv`

The hyperparameters of the selected reference run (`Liu_like_v01`) are summarized below.

TABLE 3.4. TRAINING HYPERPARAMETERS OF THE SELECTED RUN (`LIU_LIKE_V01`)

Hyperparameter	Value
Training mode	Adam
Seed (data-loader generator)	42
Epochs	8000
Train batch size	1024
Validation batch size	All validation samples
Loss	MSE
Adam learning rate	10^{-3}
Adam weight decay	0.0
LR scheduler	StepLR, step_size = 500, $\gamma = 0.5$
Early stopping	Disabled

Source: Own elaboration from run configuration copies

3.4.3. Regularization And Callbacks

PUEDE IR CAMBIANDO: se pretende mejorar las runs

Regularization in the baseline is intentionally light: dropout is disabled ($p = 0$), batch

normalization is disabled, and Adam weight decay is set to zero. In addition, feature/target normalization is turned off in the Liu-style reference setup. This design keeps the training protocol close to the reference architecture and isolates the effect of optimizer scheduling and data quality controls.

Callback management includes: (i) checkpointing of both *best* and *last* models, (ii) StepLR scheduling for Adam, and (iii) an early-stopping module available in the pipeline but disabled in the baseline experiments. When enabled, early stopping supports both standard validation loss and trimmed validation loss monitors.

From an experimental standpoint, this callback configuration improves training robustness without over-constraining the optimization path: checkpoints protect against late-epoch degradation, while the Adam-to-L-BFGS transition plus scheduler supports stable convergence. Generalization is then assessed empirically on held-out validation and test splits rather than enforced through strong explicit regularizers.

4. CALIBRATION NEURAL NETWORK FOR THE HESTON MODEL

4.1. Calibration problem formulation

- hablar sobre el concepto de generalization
- hablar sobre Differential Evolution. asi como sus argumentos e interpretación (en Liu está bien explicado)

4.2. Input representation: implied volatility surfaces

4.3. Synthetic calibration dataset

4.4. CaNN architecture

5. PHYSICS-INFORMED NEURAL PRICING UNDER THE HESTON MODEL

This chapter reformulates vanilla option pricing under the Heston model as a PDE-constrained learning problem. In contrast to the supervised ANN surrogate of Chapter 3, the objective is no longer to emulate precomputed numerical labels, but to approximate directly the option-price function through an unsupervised Physics-Informed Neural Network (PINN). The methodology follows the general PINN philosophy and is inspired by the Heston-specific formulation proposed in [5], while adapting the state representation and notation to the conventions used in the present work.

5.1. Problem formulation

In this chapter, European option pricing under Heston is posed as the approximation of the solution of the pricing PDE rather than as a supervised regression problem. The neural network outputs the option price itself, and the training objective is to enforce that this output satisfies the Heston valuation equation together with its terminal and boundary conditions. Consequently, the core information driving the optimization is not a set of external labels, but the analytical structure of the model under the risk-neutral measure.

To avoid a notational clash between the price function and the variance state, the option value will be denoted by u, while the instantaneous variance state will be denoted by v. This distinction is important because the Heston model contains both a variance process and a pricing function, and using the same symbol for both would make the PDE formulation unnecessarily confusing. The unknown function approximated by the PINN is therefore written as

$$u_\phi(\tau, m, v, \Theta, r),$$

where ϕ denotes the trainable weights of the network, $m = S/K$ is moneyness, $\tau = T - t$ is time-to-maturity, v is the current instantaneous variance, and Θ collects the structural Heston parameters.

More precisely, the calibrated tuple obtained in the previous chapter is

$$\Theta^* = (\rho, \kappa, \gamma, \bar{v}, v_0).$$

However, the role of v_0 must be distinguished from that of the other parameters. The quantities ρ, κ, γ and $\bar{v}$ define the operator of the Heston PDE, whereas v_0 is the initial condition of the variance process. In the PDE formulation, the state variable is the generic variance level v, not the fixed initial value v_0. Therefore, the parametric PINN is constructed to learn a family of Heston pricing solutions indexed by the structural parameters $(\rho, \kappa, \gamma, \bar{v}, r)$, while the current variance v_0 is used only when evaluating the trained

network at the present market state. The calibrated market valuation is thus recovered through

$$u_\phi(\tau, m, v_0, \rho^*, \kappa^*, \gamma^*, \bar{v}^*, r).$$

This point is conceptually important: during training, the PINN learns the solution over a domain in $(\tau, m, v, \rho, \kappa, \gamma, \bar{v}, r)$, whereas at inference time the current market valuation is recovered by fixing the variance coordinate to the calibrated value v_0^* and the structural parameters to the calibrated tuple Θ^*.

The network is trained on collocation points sampled in the interior and on the relevant boundaries of the pricing domain. These points are not generated by simulating full paths of the Heston variance process. Instead, they are drawn directly from a bounded domain of admissible states, for instance using Latin Hypercube Sampling (LHS), and used to evaluate the PDE residual. In that sense, the method is unsupervised: the learning signal comes from the Heston structure itself, not from externally computed price labels. This is the main methodological difference with respect to the ANN surrogate developed in Chapter 3.

At a high level, the proposed workflow is the following. First, a parametric PINN is defined to approximate the price surface over a range of maturities, moneyness levels, variance states, and Heston parameters. Second, the PINN is trained by minimizing the residual of the Heston pricing PDE together with terminal and boundary mismatches. Third, once the network is trained, the price of a contract for a calibrated market configuration is obtained by evaluating the network at the corresponding $(\tau, m, v_0^*, \Theta^*, r)$. This preserves the link with the calibration pipeline while replacing label-based supervision with model-consistent unsupervised training. In this sense, the CaNN stage remains essential: it provides the market-specific parameter tuple that is consumed by the parametric PINN at inference time, without requiring a retraining of the pricing network for each new calibration.

- hablar muy bien de los inputs (heston params optimizados y tal)
- hablar sobre el output, que es V para vanilla option
- hablar sobre greeks por autodiff ?
- hablar de nuevo un poco sobre el pipeline

5.1.1. PINN Theoretical Framework

no estoy seguro de donde poner esto, o si quitarlo incluso

A PINN can be viewed as a parametric approximation of the solution of a differential equation. In the present setting, the network u_ϕ is required to satisfy the Heston pricing PDE in the interior of the domain and to match the contractual conditions at expiry and

on selected boundaries. If $\mathcal{N}$ denotes the Heston differential operator, the target problem can be summarized abstractly as

$$\mathcal{N}[u] = 0 \quad \text{in the interior domain,} \qquad \mathcal{B}[u] = 0 \quad \text{on the boundaries,}$$

where $\mathcal{B}$ denotes the set of terminal and boundary constraints.

The PINN replaces the unknown analytical solution by a neural approximation u_ϕ and defines a residual function

$$\mathcal{R}_\phi = \mathcal{N}[u_\phi].$$

Training then consists of minimizing a composite loss built from three sources of error: the interior PDE residual, the mismatch with the terminal payoff, and the mismatch with the selected boundary conditions. In a compact notation, the loss takes the form

$$\mathcal{L}_{\text{PINN}}(\phi) = \mathcal{L}_{\text{PDE}}(\phi) + \mathcal{L}_{\text{term}}(\phi) + \mathcal{L}_{\text{bc}}(\phi).$$

This decomposition is standard in the PINN literature and matches the structure adopted in [5].

An essential ingredient of this approach is automatic differentiation. Since the Heston PDE contains first- and second-order derivatives with respect to the state variables, the network output must remain differentiable with respect to its inputs. The required quantities are obtained by differentiating the computational graph of the network rather than by using finite-difference approximations. This is particularly convenient in the present case because the same differentiable surrogate can later be reused for the computation of Greeks.

- función aproximadora
- idea de residuo de la PDE
- concepto de la loss en una PINN

5.2. Heston Pricing PDE

Under the risk-neutral measure, the Heston model is described by the coupled system dejar solo en los primeros chapters y referencialos, o poner bibliografia

$$dS_t = rS_t\,dt + \sqrt{v_t}\,S_t\,dW_t^{(1)}, \tag{5.1}$$
$$dv_t = \kappa(\bar{v} - v_t)\,dt + \gamma\sqrt{v_t}\,dW_t^{(2)}, \tag{5.2}$$

with instantaneous correlation

$$d\langle W^{(1)}, W^{(2)}\rangle_t = \rho\,dt.$$

Here r is the risk-free rate, κ is the mean-reversion speed of the variance process, $\bar{v}$ is the long-run variance level, γ is the volatility of variance, and ρ controls the correlation

between returns and variance shocks. The current variance level at valuation time is denoted by v_0, so that v_t evolves from the initial condition $v(0) = v_0$.

Let $u(t, S, v)$ denote the price of a European vanilla option. By the Feynman-Kac representation, u satisfies the Heston pricing PDE [6]

$$u_t + \frac{1}{2}vS^2u_{SS} + \rho\gamma vSu_{Sv} + \frac{1}{2}\gamma^2 vu_{vv} + rSu_S + \kappa(\bar{v} - v)u_v - ru = 0. \tag{5.3}$$

This is the continuous-time constraint that will be enforced by the PINN at interior collocation points. In the present chapter, the price function is denoted by u precisely to keep it separate from the variance state v.

For implementation purposes, it is more convenient to work with time-to-maturity $\tau = T - t$ and moneyness $m = S/K$ instead of calendar time t and spot S. Since the synthetic and market pipelines of this project use a fixed strike normalization $K = 1$, the pricing problem can be rewritten in terms of the state variables (τ, m, v). Defining the same pricing function in transformed coordinates, the PDE becomes

$$u_\tau = \frac{1}{2}vm^2u_{mm} + \rho\gamma vmu_{mv} + \frac{1}{2}\gamma^2 vu_{vv} + rmu_m + \kappa(\bar{v} - v)u_v - ru. \tag{5.4}$$

This is the formulation that will be used to define the PDE residual in the training loss. It remains fully equivalent to the original Heston pricing equation, but it is better aligned with the representation used in the rest of the thesis.

For a European put option with normalized strike $K = 1$, the terminal condition at expiry is

$$u(0, m, v) = (1 - m)^+. \tag{5.5}$$

The payoff does not depend explicitly on the variance state, which means that the terminal condition is imposed uniformly over the v-dimension of the domain. In addition, a natural lower boundary condition is obtained at zero underlying value:

$$u(\tau, 0, v) = e^{-r\tau}. \tag{5.6}$$

For a general strike K, this condition would read $u(\tau, 0, v) = Ke^{-r\tau}$. In a first PINN specification, these two constraints are sufficient to reproduce the structure used in [5]: one term of the loss is attached to the interior PDE residual, one to the terminal payoff, and one to the lower boundary.

The far-field behavior as $m \to \infty$ for a put, namely $u(\tau, m, v) \to 0$, can also be incorporated if needed. However, to keep the initial implementation simple, the first version of the model will explicitly enforce only the terminal condition and the lower boundary, while controlling the remaining behavior through the bounded sampling domain in (τ, m, v) and the interior residual. This choice is consistent with the objective of building a minimal but coherent unsupervised PINN before adding more elaborate constraints.

- presentar la PDE de heston y hablar sobre ella

- definir la terminal condition payoff (depende de put/call !)
- hablar de las boundary conditions y cuales imponemos

5.3. Neural Network Architecture

en general parecido a chapter 3

5.3.1. Model Class

5.3.2. Architecture Details

5.3.3. Model Inputs and Outputs

5.4. Training Procedure

5.4.1. Loss Function

The PINN is trained by minimizing a composite loss that enforces the pricing PDE in the interior of the domain together with the terminal payoff and the lower boundary condition. Let $u_\phi(\tau, m, v, \rho, \kappa, \gamma, \bar{v}, r)$ denote the output of the neural network. From the transformed Heston PDE introduced above, the residual at an interior point is defined as

$$\mathcal{R}_\phi = u_\tau - \frac{1}{2}vm^2u_{mm} - \rho\gamma vmu_{mv} - \frac{1}{2}\gamma^2 vu_{vv} - rmu_m - \kappa(\bar{v} - v)u_v + ru, \tag{5.7}$$

where all derivatives are evaluated by automatic differentiation on the computational graph of the network. In the ideal case, the exact pricing function would satisfy $\mathcal{R}_\phi = 0$ for every admissible interior point.

Let $\mathcal{S}_D = \{z_j\}_{j=1}^{N_D}$ denote the set of interior collocation points, with

$$z_j = (\tau_j, m_j, v_j, \rho_j, \kappa_j, \gamma_j, \bar{v}_j, r_j).$$

The interior loss is the mean squared residual

$$\mathcal{L}_{\text{PDE}}(\phi) = \frac{1}{N_D}\sum_{j=1}^{N_D} \mathcal{R}_\phi(z_j)^2. \tag{5.8}$$

On the terminal set $\mathcal{S}_T = \{z_k^T\}_{k=1}^{N_T}$, where $\tau = 0$, the network must match the payoff of the European put. The corresponding error is

$$e_k^T(\phi) = u_\phi(0, m_k, v_k, \rho_k, \kappa_k, \gamma_k, \bar{v}_k, r_k) - (1 - m_k)^+, \tag{5.9}$$

and the terminal loss is defined by

$$\mathcal{L}_{\text{term}}(\phi) = \frac{1}{N_T}\sum_{k=1}^{N_T} \left(e_k^T(\phi)\right)^2. \tag{5.10}$$

On the lower boundary set $\mathcal{S}_{\text{low}} = \{z_l^{\text{low}}\}_{l=1}^{N_{\text{low}}}$, where $m = 0$, the exact put price is the discounted strike. Since the implementation uses the normalized strike $K = 1$, the lower-boundary error is

$$e_l^{\text{low}}(\phi) = u_\phi(\tau_l, 0, v_l, \rho_l, \kappa_l, \gamma_l, \bar{v}_l, r_l) - e^{-r_l \tau_l}, \tag{5.11}$$

and the associated loss is

$$\mathcal{L}_{\text{bc}}(\phi) = \frac{1}{N_{\text{low}}} \sum_{l=1}^{N_{\text{low}}} \left(e_l^{\text{low}}(\phi)\right)^2. \tag{5.12}$$

The total training objective is then written as

$$\mathcal{L}_{\text{PINN}}(\phi) = \mathcal{L}_{\text{PDE}}(\phi) + \mathcal{L}_{\text{term}}(\phi) + \mathcal{L}_{\text{bc}}(\phi). \tag{5.13}$$

For implementation, it is convenient to keep an explicit weighted form,

$$\mathcal{L}_{\text{PINN}}(\phi) = \lambda_{\text{PDE}}\mathcal{L}_{\text{PDE}}(\phi) + \lambda_{\text{term}}\mathcal{L}_{\text{term}}(\phi) + \lambda_{\text{low}}\mathcal{L}_{\text{bc}}(\phi), \tag{5.14}$$

with non-negative coefficients $(\lambda_{\text{PDE}}, \lambda_{\text{term}}, \lambda_{\text{low}})$ that can be tuned if one component dominates the optimization. In the baseline specification of this thesis, all three weights are initialized to one, i.e. $(1, 1, 1)$, following the simplest formulation in [5].

A practical point is that the PDE term requires second-order derivatives with respect to inputs. Therefore, the automatic-differentiation graph must be kept when computing first derivatives, so that mixed and second derivatives such as u_{mv} and u_{vv} can be obtained consistently in the same computational graph. This is the key implementation detail that enables direct optimization of the PDE residual without finite-difference approximations.

5.4.2. Optimizer and Hyperparameters

The optimization follows a two-stage strategy that is common in PINN practice: an initial stochastic phase with Adam, followed by a quasi-Newton refinement with L-BFGS. In this work, this strategy is implemented as a mixed schedule in which Adam is used during the first part of training and L-BFGS in the second part, while a pure Adam or pure L-BFGS run remains available as an ablation.

Since the three loss terms are evaluated on different sample sets, each epoch uses three mini-batches: one from $\mathcal{S}_D$ (interior), one from $\mathcal{S}_T$ (terminal), and one from $\mathcal{S}_{\text{low}}$ (lower boundary). This allows different batch sizes for interior and boundary constraints and keeps the training loop aligned with the composite objective.

The reference hyperparameter block for the first implementation includes: learning rates and optimizer-specific settings, the Adam-to-L-BFGS switching epoch in mixed mode, batch sizes for interior and boundary sets, and the triplet of loss weights $(\lambda_{\text{PDE}}, \lambda_{\text{term}}, \lambda_{\text{low}})$.

For diagnostics, the training run stores epoch-wise train/validation curves for the total weighted loss and for each component separately. In addition, a two-dimensional error

map over (m, τ) is produced from validation interior points by averaging $|\mathcal{R}_\phi|$ in bins. This map helps identify whether specific regions of the contract domain remain systematically underfitted even when the global loss decreases.

5.4.3. Sampling strategy

Training requires three different sample sets, each associated with one component of the loss. As in the general PINN methodology, these sets are not built from labeled prices but from points drawn directly in the state-parameter domain. The first set corresponds to the interior of the PDE domain, the second to the terminal boundary at expiry, and the third to the lower boundary at zero underlying value.

The interior domain is defined over bounded intervals for the state variables and for the Heston parameters:

$$\tau \in [0, \tau_{\max}], \qquad m \in [m_{\min}, m_{\max}], \qquad v \in [v_{\min}, v_{\max}],$$

combined with admissible ranges for $(\rho, \kappa, \gamma, \bar{v}, r)$. These parameter intervals should remain consistent with the calibration regime studied in the previous chapters, since the purpose of the parametric PINN is precisely to generalize over the family of Heston configurations that may be produced by the CaNN stage. In particular, the v-dimension is sampled explicitly as a state coordinate of the PDE. This is a crucial point: the PINN is trained on generic variance states v, whereas the calibrated initial variance v_0 is only used later, when evaluating the trained network at the current market state.

To obtain a representative coverage of this multidimensional domain with a controlled number of points, the baseline sampling method will be Latin Hypercube Sampling (LHS). Compared with naive Monte Carlo sampling, LHS provides a more regular coverage of each marginal dimension and reduces the risk of leaving large regions of the domain underexplored. In the present setting, this is particularly useful because the PINN must generalize simultaneously over state variables and model parameters.

For the interior set $\mathcal{S}_D$, each sampled point has the form

$$z_j = (\tau_j, m_j, v_j, \rho_j, \kappa_j, \gamma_j, \bar{v}_j, r_j), \qquad j = 1, \ldots, N_D.$$

These are the collocation points at which the PDE residual is evaluated. For the terminal set $\mathcal{S}_T$, points are sampled in the subdomain

$$\tau = 0, \qquad m \in [m_{\min}, m_{\max}], \qquad v \in [v_{\min}, v_{\max}],$$

again jointly with the Heston parameters. This set is used to impose the payoff condition. For the lower-boundary set $\mathcal{S}_{\text{low}}$, the sampled points satisfy

$$m = 0, \qquad \tau \in [0, \tau_{\max}], \qquad v \in [v_{\min}, v_{\max}],$$

with the same parameter ranges, and are used to enforce the discounted-strike boundary condition.

From an implementation viewpoint, the three sets can be regenerated dynamically during training or precomputed once and reused across epochs. A dynamic resampling strategy may improve coverage of the domain and reduce overfitting to a fixed cloud of collocation points, whereas a static design simplifies reproducibility and debugging. The initial implementation will prioritize simplicity and reproducibility; therefore, the first reference version will rely on fixed sample sets generated before the training loop. Dynamic resampling can later be evaluated as an extension if it leads to more stable convergence.

This sampling strategy preserves the fully unsupervised nature of the method. No prices from COS, FFT, or Monte Carlo are required to build the training objective. Numerical pricing methods remain useful for ex-post validation and benchmarking, but not as a source of labels for fitting the PINN itself.

5.4.4. Input scaling

Probablemente cambie !!

Although the pricing problem is formulated in physical variables, the actual optimization of the PINN benefits from an affine rescaling of its inputs. The motivation is purely numerical: the state variables (τ, m, v) and the Heston parameters do not share comparable magnitudes, and this imbalance may lead to poorly conditioned gradients and unstable convergence. As emphasized in [5], centering and scaling the inputs helps mitigate optimization issues such as vanishing or highly unbalanced gradients.

In the present work, the first implementation will use a simple affine transformation of each input variable,

$$\tilde{x} = a_x + b_x x,$$

applied componentwise. At minimum, this transformation will be used for the PDE state variables (τ, m, v). If the network is later extended to a fully parametric PINN over $(\rho, \kappa, \gamma, \bar{v}, r)$, the same rescaling principle can be applied to those parameters as well. The exact coefficients (a_x, b_x) may be defined either from prescribed domain bounds or from summary statistics of the sampled training points.

Once the network is expressed in scaled coordinates, all derivatives entering the PDE residual must be interpreted with respect to the transformed variables. In practice, this does not alter the mathematical structure of the approach: the residual is still computed by automatic differentiation, and the change of variables is handled through the computational graph and the chain rule. Therefore, scaling is introduced as a numerical stabilization device rather than as a modification of the underlying financial model.

5.5. Greeks Computation

5.5.1. Defition of Greeks

- definir greeks
- como obtenerlas derivando la red

5.5.2. Automatic Differentiation of the PINN

- definir autodiff
- comparar con otros metodos como finite difs
- interpretación

6. NEURAL NETWORK PERFORMANCE AND BENCHMARKING

6.1. Pricing Accuracy

6.2. Implied Volatility Accuracy

6.3. Computational Efficiency

6.4. Generalization And Robustness

7. CONCLUSION AND FUTURE WORK

7.1. Market Data

7.2. Exotic Derivatives

7.3. Greeks And Hedging

7.4. Model Extension

7.5. Machine Learning Improvements

BIBLIOGRAPHY

[1] S. Liu, A. Borovykh, L. A. Grzelak, and C. W. Oosterlee, "A neural network-based framework for financial model calibration," en, *Journal of Mathematics in Industry*, vol. 9, no. 1, p. 9, Dec. 2019. DOI: 10.1186/s13362-019-0066-7. Accessed: Oct. 21, 2025. [Online]. Available: https://mathematicsinindustry.springeropen.com/articles/10.1186/s13362-019-0066-7.

[2] M. D. Mckay, R. J. Beckman, and W. J. Conover, "A Comparison of Three Methods for Selecting Values of Input Variables in the Analysis of Output From a Computer Code," en, *Technometrics*, vol. 42, no. 1, pp. 55–61, Feb. 2000. DOI: 10.1080/00401706.2000.10485979. Accessed: Mar. 21, 2026. [Online]. Available: http://www.tandfonline.com/doi/abs/10.1080/00401706.2000.10485979.

[3] R. B. Gramacy, *Surrogates: Gaussian Process Modeling, Design, and Optimization for the Applied Sciences*. Chapman and Hall/CRC, 2020.

[4] W. H. Press, S. A. Teukolsky, W. T. Vetterling, and B. P. Flannery, *Numerical Recipes: The Art of Scientific Computing*, 3rd ed. Cambridge University Press, 2007.

[5] D. Hainaut and A. Casas, "Option pricing in the heston model with physics inspired neural networks," en, *Annals of Finance*, vol. 20, no. 3, pp. 353–376, 2024. DOI: 10.1007/s10436-024-00452-7. [Online]. Available: https://doi.org/10.1007/s10436-024-00452-7.

[6] S. L. Heston, "A Closed-Form Solution for Options with Stochastic Volatility with Applications to Bond and Currency Options," en,

APPENDIX A: ANALYSIS OF THE COS METHOD ERROR

no se si dejarlo como apéndice o meterlo en alguna seccion

For the COS method, we have three sources of error:

- The integration range truncation error, ε_1.
- The series truncation error on $[a, b]$, ε_2.
- The error related to approximating $\bar{A}_k(x)$ by $\bar{F}_k(x)$, ε_3 (poner ref a alguna ecuacion).

It is important to note that there is no error in the payoff coefficient H_k for plain vanilla European options.

If the integration range $[a, b]$ is chosen sufficiently large, the overall error is dominated by ε_2 (TODO: argumentarlo). Also, ε_2 converges exponentially if $f_X(x) \in C^\infty[a, b]$ (TODO: justificarlo tamb). Otherwise, converges only algebraically.

To choose an optimal integration range we will use the expression (TODO: ref a Oorsterlee)

$$[a, b] = \left[(x + \zeta_1) - L\sqrt{\zeta_2 + \sqrt{\zeta_4}},\ (x + \zeta_1) + L\sqrt{\zeta_2 + \sqrt{\zeta_4}} \right], \tag{7.1}$$

where $L \in [6, 12]$ is a parameter that depends on the tolerance we want to have, and ζ_i are the cumulants, computed using the expression (2.6).

APPENDIX B: CALIBRATION OF THE HESTON MODEL

TODO: